AI-Driven Local Oscillator Synthesizer Built on a 45 nm CMOS Charge-Pump PLL

Implementation report: completed baseline work and the plan for AI integration

Cadence Virtuoso | Spectre / Spectre RF | Python (PyTorch) co-simulation

---

Prepared by: [Team / Names] | Date: October 2026 | Reviewer: [Supervisor]

Contents

1. Executive Summary
2. Baseline PLL: Work Completed
3. Gap Analysis: Baseline vs. AI Requirements
4. Why AI: The Classical PLL Trade-off
5. Phase 1: Characterization and Dynamic Locking
6. Phase 2: AI Architecture and Implementation
7. Phase 3: Layout, Post-Layout and PVT Validation
8. Schedule and Milestones
9. Risks and Mitigation
10. Expected Outcomes and Conclusion

1. Executive Summary

We have taken a full-custom charge-pump PLL (CPPLL) in a 45 nm CMOS process from blank schematic to an integrated, simulating top-level design in Cadence Virtuoso. Every block, from the phase-frequency detector (PFD) through the current-starved VCO and TSPC divider, is built at transistor or gate level, and the first Spectre transient run over 1.5 µs converged cleanly and settled to a stable operating point with the control voltage Vc holding steady.

That run used a fixed reference, so it validated biasing and block interaction rather than dynamic lock. The next stage is therefore twofold. First, we characterize the baseline properly: a clocked reference, the VCO tuning curve $K_{VCO}(Vc)$, final loop filter values, and phase noise. Second, we build on that baseline with an AI layer that attacks the classic PLL compromise between lock time, jitter and reference spurs.

The AI layer has three parts: a small feedforward neural network that predicts the steady-state control voltage and pre-charges the loop filter so the loop starts almost on frequency; a reinforcement-learning (or adaptive) tuner that adjusts the loop filter resistor and charge-pump trim; and a background calibration loop that cancels charge-pump current mismatch and the reference spur it produces. Algorithms are developed in a Python–Spectre co-simulation environment, then followed by layout, parasitic extraction and a full PVT matrix.

Headline targets

Metric	Baseline (estimated, to be measured)	AI-augmented target
Lock acquisition time	~1.5 to 5.0 µs	< 250 ns
Integrated RMS jitter (10 kHz–100 MHz)	~1.2 to 2.5 ps	< 350 fs
Reference spur	−30 to −40 dBc	< −58 dBc
Maturity	Schematic-level bring-up	DRC/LVS-clean layout, post-layout verified

The baseline figures are expectations for an uncompensated design of this type. Part of Phase 1 is replacing them with measured numbers, so the comparison in the final paper rests on our own data.

2. Baseline PLL: Work Completed

The baseline is a conventional type-II charge-pump PLL. The reference and the divided VCO output feed the PFD; its up and down pulses steer the charge pump; the pump current is integrated by a passive second-order filter into Vc; and Vc sets the VCO frequency.

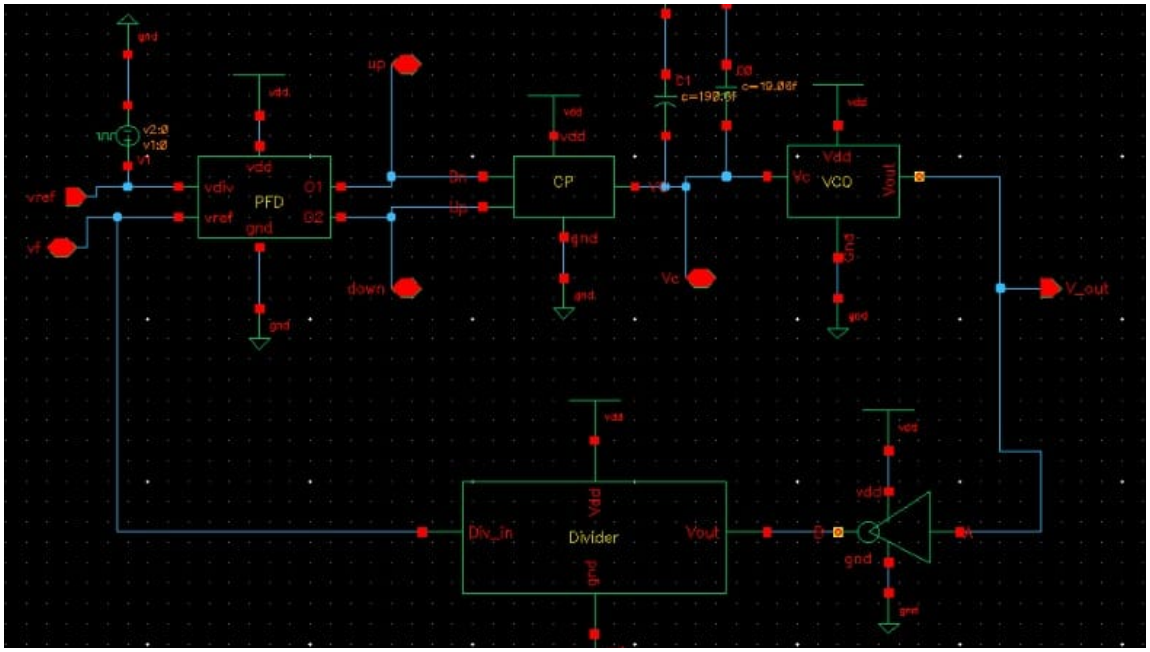


Figure 1. Baseline PLL top-level schematic (PFD, charge pump, loop filter, VCO, divider and feedback inverter).

2.1 Block-level summary

Block	Implementation	Notes
PFD	Two resettable D flip-flops plus reset gate; each flip-flop built from four NOR gates	Dead-zone-free up/down pulses; self-contained, no library cells
Charge pump	PMOS pull-up / NMOS pull-down branches, 15 µA mirrored bias, 45 nm devices at W = 120 nm, L = 45 nm	Inverter on the Up path sets switch polarity
Loop filter	Passive second order: R1 in series with C1, C2 shunt	Final values to be set after K_{VCO} extraction
VCO	29-instance current-starved ring with mirror-based bias and output stage	Frequency range not yet measured
Divider	Six-stage TSPC flip-flop chain with a ratio-shaping gate	Single clock phase, low transistor count; planned ratio ÷64 (confirm)
Feedback inverter	CMOS inverter, W = 120 nm, L = 45 nm	Sharpens edges into the divider and detector

2.2 First transient bring-up

We probed vref, vf, up, down, Vc, V_out and the branch currents of one PMOS and one NMOS device in the charge pump over 1.5 µs. The run converged without trouble, V_out settled within a few nanoseconds and held, Vc stayed flat, and up/down and pump currents showed regular small ripple, which confirms the detector-to-pump handoff is active. The viewer autoscales each trace, so those ripples are fine structure on a steady bias, not large swings.

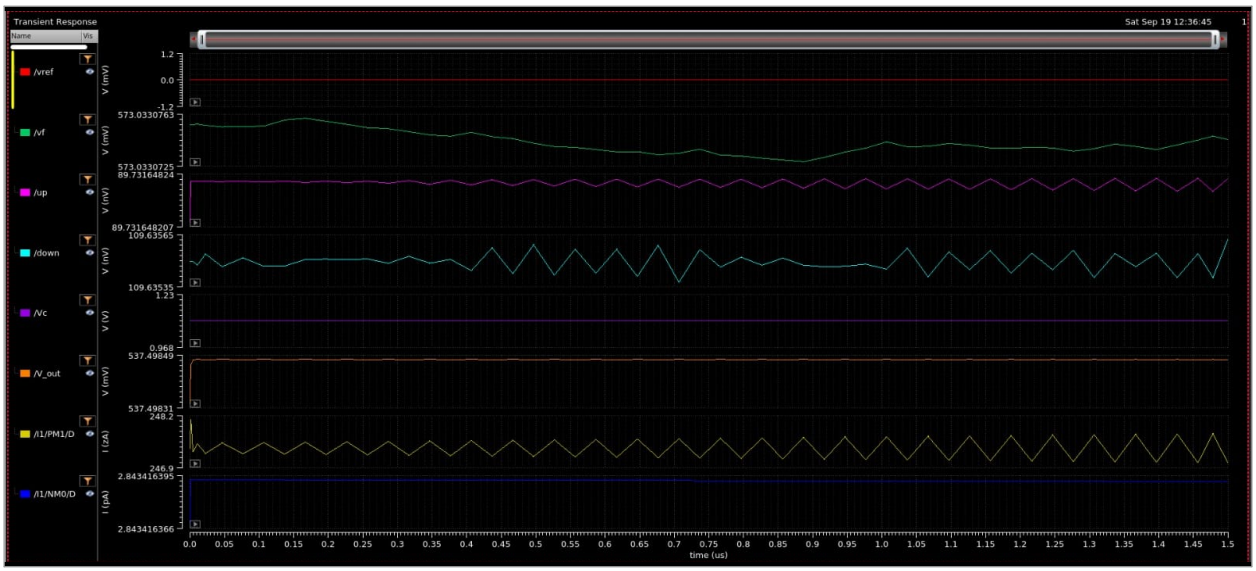


Figure 2. Baseline transient response (0 to 1.5 µs) with a fixed reference.

What this run does not tell us is how the loop behaves with a toggling reference: pull-in, lock time, cycle slipping and noise are all still open. These are exactly the items the AI work must be measured against, so they come first.

3. Gap Analysis: Baseline vs. AI Requirements

Before adding intelligence to a loop, it helps to say plainly what the baseline lacks. The table lists each sub-block, its present state, and the limitation the AI layer is meant to address.

Sub-block / metric	Current status (45 nm)	Limitation the AI layer targets
Reference stimulus	Fixed (non-toggling) reference	Needs clocked source to extract pull-in range and lock acquisition
Frequency acquisition	Classical negative-feedback settling	Slew-rate limited; prone to cycle slipping on channel hops
VCO	29-instance current-starved ring; range unmeasured	Uncharacterized K_{VCO} non-linearity erodes phase margin
Charge pump matching	Symmetric W/L = 120 nm / 45 nm; ΔI_{CP} uncalibrated	Channel-length modulation gives current mismatch and reference spurs
Loop filter	R1, C1, C2 not yet finalized; static bandwidth	Fixed bandwidth forces speed-versus-jitter compromise
Verification scope	Nominal corner, 27 °C, 1.5 µs window	No PVT robustness, transient noise or parasitics yet

4. Why AI: The Classical PLL Trade-off

A conventional PLL is a negotiation. A wide loop bandwidth locks fast and suppresses VCO noise, but passes more reference noise and spur energy. A narrow bandwidth cleans up the output and lowers spurs, but locks slowly and is more exposed to cycle slips. Charge-pump mismatch makes this worse, because any difference between up and down current creates a static phase offset and a periodic disturbance on Vc at the reference rate.

Designers usually settle on one compromise bandwidth. That setting is only optimal for one corner, one temperature and one channel. The AI approach splits the problem across time instead of compromising: a fast, feedforward action gets the loop to the right frequency quickly, a narrow, clean loop takes over for steady-state, and background calibration removes the mismatch that causes spurs. Each piece is small enough to be realistic in silicon.

4.1 Division of labour

- **Feedforward ANN:** handles the start-up and channel-hop transient, where speed matters most.
- **RL / adaptive tuner:** handles bandwidth and damping, trading speed for jitter as the situation requires.
- **Background calibration:** handles charge-pump mismatch, which sets the reference spur floor.

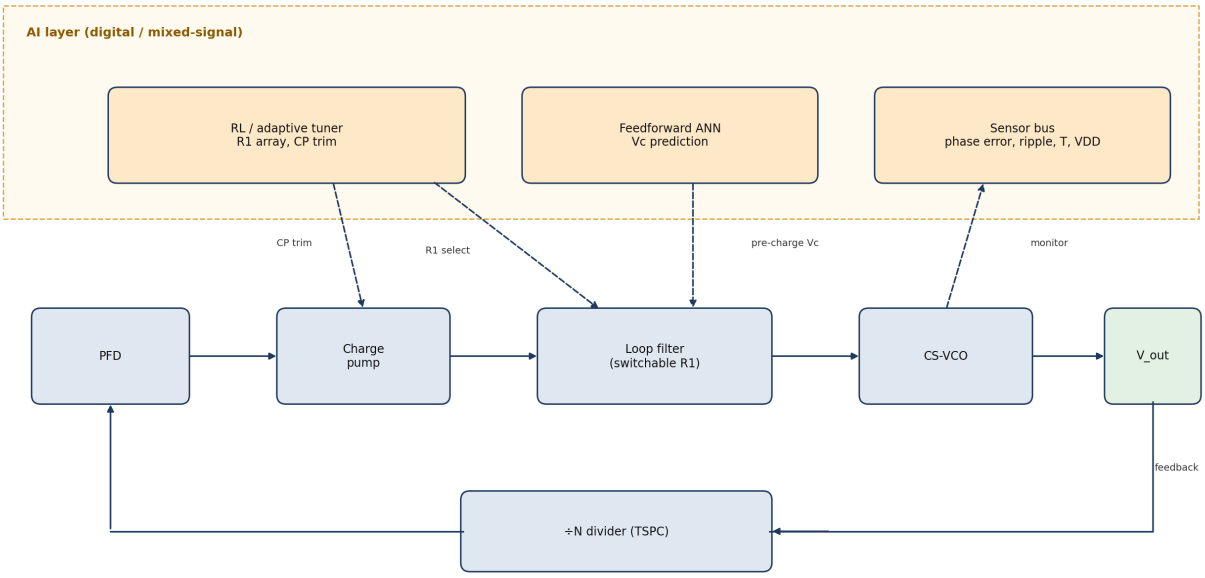


Figure 3. AI-augmented PLL: the analog loop is unchanged; the AI layer injects, selects and trims.

5. Phase 1: Characterization and Dynamic Locking

Phase 1 produces the measured baseline and the data the ANN will later be trained on. It is deliberately done before any AI work so every improvement can be attributed to the AI layer and not to retuning.

5.1 Clocked reference and acquisition profiling

The fixed source at vref is replaced by an ideal pulse source at the target reference frequency, with realistic edges ($t_r = t_f$ = 20 ps) and duty cycle varied from 45% to 55% to check that the PFD does not depend on a perfect 50% clock. Transients of 10 to 50 µs are run from several initial control voltages (Vc(0) = 0 V, 0.6 V, 1.2 V).

- Pull-in time T_p and final lock time T_L, defined inside a ±0.1% frequency settling envelope.

- Hold-in range and pull-in range, found by stepping the reference frequency until lock is lost or fails to acquire.
- The differential trace V(up) − V(down) at the charge-pump input, to confirm the NOR-based flip-flops avoid dead zones and metastability when reference and feedback edges collide.

5.2 VCO tuning curve

The VCO is decoupled from the loop filter and Vc is swept from 0 V to V_{DD} = 1.2 V in 10 mV steps at 27 °C. The output frequency at each point gives f_{out}(Vc), and the local sensitivity follows from its derivative:

$$K_{VCO}(Vc) = 2\pi \cdot d\, f_{out} / d\, Vc \quad [rad/(s{\cdot}V)]$$

The region where K_{VCO} stays within ±10% of its mean defines the usable control range, Vc,min to Vc,max. Operating inside it prevents the loop gain from wandering enough to threaten stability. The same curve is the ground truth the ANN will learn to invert.

5.3 Loop filter synthesis

With the average K_{VCO} in hand, the second-order filter is sized from a chosen natural frequency ω_n, damping ζ and divide ratio N. Using the phase detector gain $I_{CP}/2\pi$:

$$C_1 = I_{CP}\, K_{VCO} / (2\pi\, N\, \omega_n^{\,2}), \quad R_1 = 2\zeta / (\omega_n\, C_1), \quad C_2 \approx C_1 / 10$$

The target is an open-loop phase margin of at least 62° with a loop bandwidth near one tenth of the reference frequency. These closed-form values are a starting point; AC and stability analysis on the real netlist decides the final R1, C1 and C2, which also fills the placeholder left in the earlier progress report.

5.4 Noise and jitter baseline

Spectre RF periodic noise (pnoise, jitter mode) is run over a 10 kHz to 100 MHz offset range to get the phase-noise spectrum $S_\varphi(f)$ and integrated RMS jitter σ_t of the baseline PLL. A transient-noise run with the clocked reference provides a cross-check, and reference spur level is read from the output spectrum once locked. These three numbers (lock time, jitter, spur) are the baseline column in the final comparison.

6. Phase 2: AI Architecture and Implementation

The AI layer wraps the existing analog loop without changing its fundamental structure. If the AI layer is disabled, the circuit falls back to a plain, stable PLL, which is important both for verification and for robustness.

6.1 Feedforward ANN for fast frequency acquisition

When a channel hop is requested, a classical loop must slew Vc from its old value to the new one, limited by pump current and filter capacitance. The ANN skips most of that slew by predicting where Vc will end up and putting it there directly.

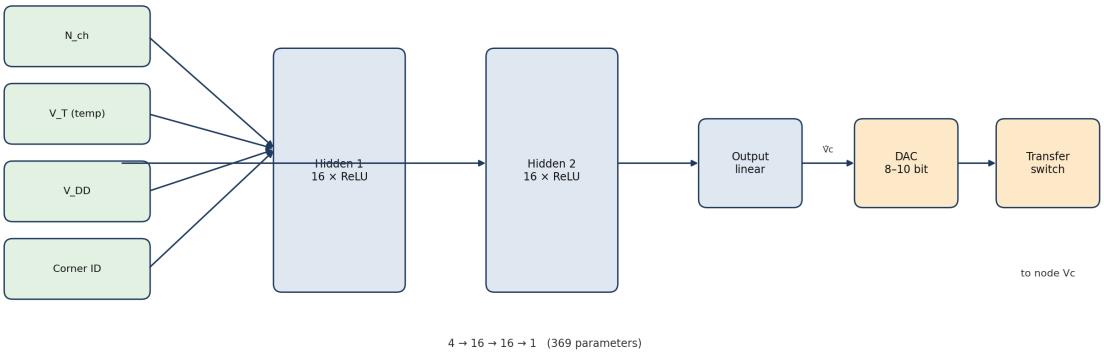


Figure 4. ANN predictor and pre-charge path: four inputs, two hidden layers of 16 ReLU neurons, one linear output.

Network and inputs

The model is a multilayer perceptron with a 4-neuron input layer, two hidden layers of 16 ReLU neurons, and a single linear output. That is 369 trainable parameters, small enough for a compact fixed-point implementation. The inputs are the channel division word N_{ch}, an on-chip PTAT temperature voltage V_T, the supply V_{DD}, and an estimated process-corner ID. The output is the predicted control voltage:

$$\hat{V}c = f_{ANN}(N_{ch}, T, V_{DD}, Corner)$$

Training data and accuracy requirement

Training data comes straight from Phase 1: for each combination of channel, temperature, supply and corner, the steady-state Vc is taken from the VCO tuning curve or from a long locked transient. The accuracy the network needs follows from the loop itself. If the residual error is ΔVc, the starting frequency error is about $K_{VCO}\cdot\Delta Vc/2\pi$, and this must sit well inside the pull-in range so the analog loop finishes the job without slipping a cycle. That requirement, not a generic error metric, sets the training loss target and the DAC resolution (an 8-bit DAC over 1.2 V gives ~4.7 mV steps; 10 bits gives ~1.2 mV).

Hand-off sequence

- On a hop command the ANN infers V̂c from the current inputs.
- A transmission-gate switch connects a low-power capacitive DAC to node Vc, pre-charging the loop filter to V̂c within two to three reference cycles.
- The switch opens and the analog loop takes over with near-zero frequency and phase error, so the pull-in phase is essentially skipped.

Two details need care in the circuit: charge injection from the transfer switch when it opens, which must be small compared with the residual error budget, and the DAC drive strength, which has to charge C1 + C2 within the allotted cycles. Both are checked in simulation in Milestone 3. For deployment the weights are quantized (8-bit is the starting point) and quantization-aware training keeps the accuracy loss small.

6.2 Reinforcement-learning tuner for loop optimization

The second AI element adapts the loop dynamics. During acquisition a wider bandwidth is useful; once locked, a narrower one gives lower jitter and spur. An RL policy learns when to switch between them.

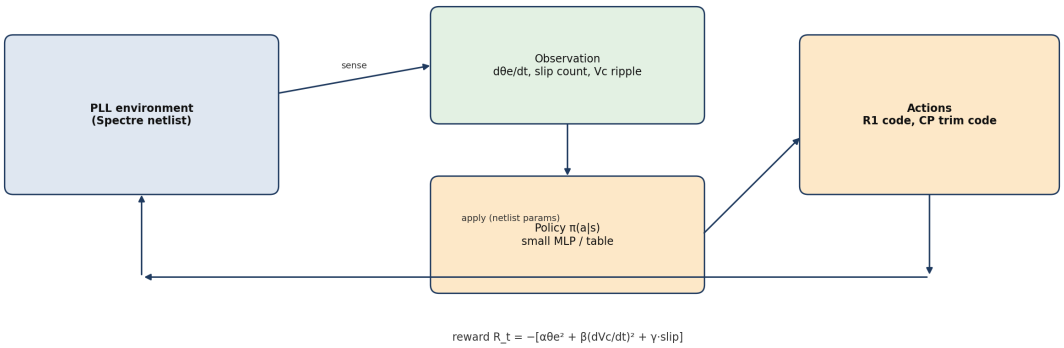


Figure 5. RL control loop: observations from the PLL drive a policy that sets R1 and charge-pump trim codes.

Element	Definition
Observation S_t	Phase-error derivative $d\theta_e/dt$, cycle-slip events, ripple amplitude on Vc
Action A_t	Binary-weighted resistor array in the loop filter ($R_{1,eff}$ within ±50% of nominal) and digital trim words for the charge-pump bias
Reward R_t	$-[\ \alpha\lvert\theta_e(t)\rvert^2 + \beta\ (dVc/dt)^2 + \gamma\cdot I_{slip}\ ]$, with α, β, γ scaling factors and I_{slip} penalizing cycle slips
Training	Offline in the co-simulation environment across corners, hops and noise seeds

A practical point on hardware: running a full deep-RL agent on-chip is not the goal. The agent is trained offline in Python, and what ships is a compact policy (a small network or a lookup table) acting on a few coarse observations through a discrete action set. A conventional rule-based adaptive controller is kept as a fallback and as a comparison, so the paper can say clearly how much the learned policy adds beyond a hand-tuned schedule.

6.3 Online charge-pump mismatch calibration

Any mismatch between pull-up and pull-down current, $\Delta I_{CP} = I_{up} - I_{down}$, produces a static phase offset and a ripple on Vc at the reference rate, which appears as a spur at f_{ref} on the output. In the locked state the ripple amplitude is a direct measure of the mismatch.

A background digital loop monitors that ripple with a sub-sampling detector and adjusts auxiliary 4-bit current DACs embedded in the charge-pump mirrors, using a gradient-descent (sign-sign LMS style) update until ripple, and hence ΔI_{CP}, is minimized. Because this loop is slow and operates only in lock, it does not interfere with acquisition. Its resolution is bounded by the 4-bit trim step, which sets the achievable spur floor and is part of what Milestone 4 measures.

6.4 Co-simulation environment

Algorithm development needs a closed loop between the circuit simulator and the learning code. Spectre runs the PLL transient and periodically passes probe voltages (vref, vf, up, down, Vc) through an OCEAN script and a named pipe or shared-memory channel to a Python/PyTorch process. Python computes actions and sends back updated instance parameters (R1 code, trim words, DAC value) that Spectre applies at the next step.

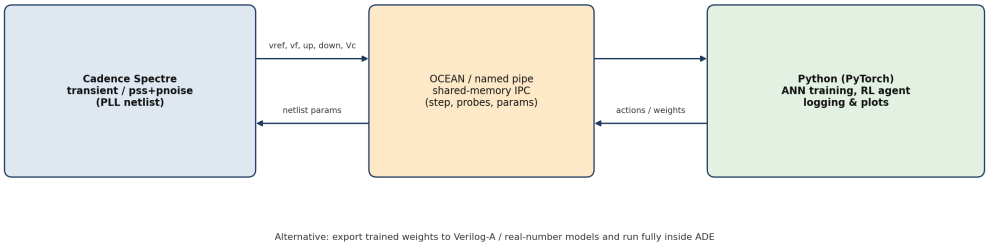


Figure 6. Spectre–Python co-simulation: probes out, actions and weights back.

Co-simulation is flexible but slow, so we keep a second route ready: export the trained weights to Verilog-A or real-number models, so the whole AI-plus-PLL system runs inside the normal Virtuoso flow and can be reused for corner sweeps and Monte Carlo without the Python bridge.

6.5 Evaluation methodology

To keep the baseline-versus-AI comparison fair, every metric has one definition and one testbench, used for both versions of the design. Lock time is measured from the hop command or reference start until the output frequency stays inside the ±0.1% envelope. Jitter is the integrated RMS value from pnoise over 10 kHz to 100 MHz. Spur level is read from the locked output spectrum at f_{ref} offset, relative to the carrier.

Each result is reported across the same set of corners and temperatures, with the AI layer enabled and disabled in the same netlist. Where the learned policy is compared against a hand-tuned rule, both see identical noise seeds. Failures, such as a cycle slip or a missed lock, are counted and reported along with the successes, since a fast-lock scheme that occasionally fails to lock is worse than a slow one.

Metric	Definition	Tool
Lock time T_L	Time to enter and stay in the ±0.1% frequency envelope	Spectre transient + OCEAN post-processing
Pull-in / hold-in range	Largest reference offset that still acquires / stays locked	Transient sweeps over reference frequency
RMS jitter σ_t	Integral of $S_\varphi(f)$ over 10 kHz–100 MHz	Spectre RF pnoise (jitter mode)
Reference spur	Spectral line at f_{ref} offset in dBc	FFT of locked V_out
Cycle-slip count	Slip events per hop across all trials	Phase-error trace analysis
AI overhead	Added power, area and supply noise from the AI layer	Power reports, floorplan area

7. Phase 3: Layout, Post-Layout and PVT Validation

Schematic gains only count if they survive the physical design. Phase 3 builds a matched, isolated layout, extracts parasitics, and re-runs the key tests across the industrial corner matrix.

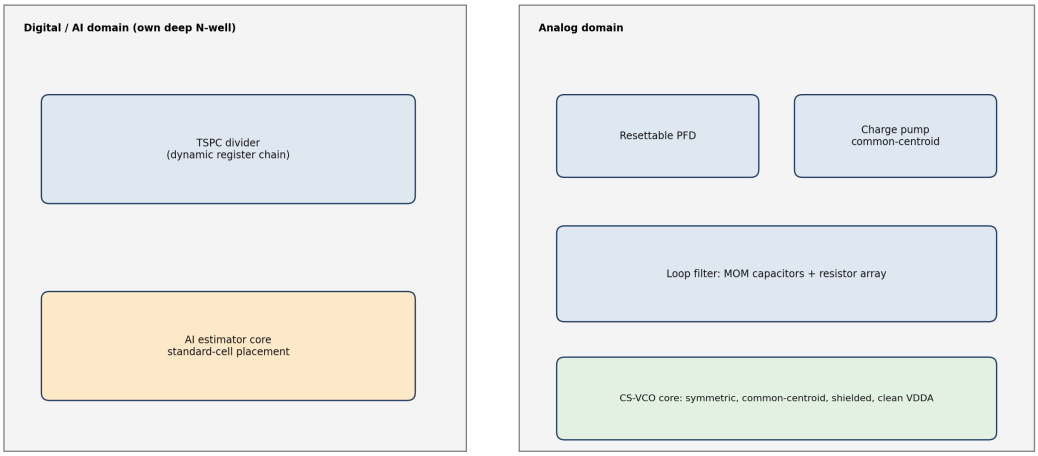


Figure 7. Planned floorplan: separate deep N-well domains for switching digital/AI logic and sensitive analog blocks.

7.1 Layout strategy

- **Isolation:** keep the VCO and loop filter in deep N-wells, away from the switching activity of the TSPC divider, PFD logic and AI core.
- **Matching:** draw the charge-pump mirrors and VCO current-starving stages as multi-finger, common-centroid structures with perimeter dummies to cancel thermal and lithographic gradients.
- **Capacitors:** build C1 and C2 as interdigitated MOM capacitors to limit substrate loss.
- **Symmetry:** balance length and parasitic capacitance on up, down and feedback paths to avoid asymmetric charge-injection spikes.

7.2 Extraction and post-layout simulation

After DRC and LVS (Pegasus or Calibre), a full R+C+CC extraction feeds post-layout transients. The checks are the drop in maximum VCO frequency, the shift in filter zero and pole from interconnect capacitance, and loop stability, with a post-layout phase margin of at least 55° required.

7.3 PVT and robustness matrix

Variable	Corners / range	Pass criterion
Process	TT, SS, FF, SF, FS	Lock retained at all corners
Supply	V_{DD} ±10% (1.08 V to 1.32 V)	K_{VCO} drift < 15% with dynamic compensation
Temperature	−40 °C to +125 °C	Integrated RMS jitter degradation < 10%
Substrate noise	50 mV_{pp} sinusoid at 100 MHz into bulk	Spur attenuation > 45 dBc

Monte Carlo runs over process and mismatch complete the picture and are the main test of the calibration loop, since mismatch is exactly what it is meant to remove. The ANN is also checked here: it must stay accurate when its sensor inputs are noisy and when it meets corners it was trained on only sparsely.

8. Schedule and Milestones

The work is organized as six milestones over fifteen weeks, ending in a four-page conference paper.

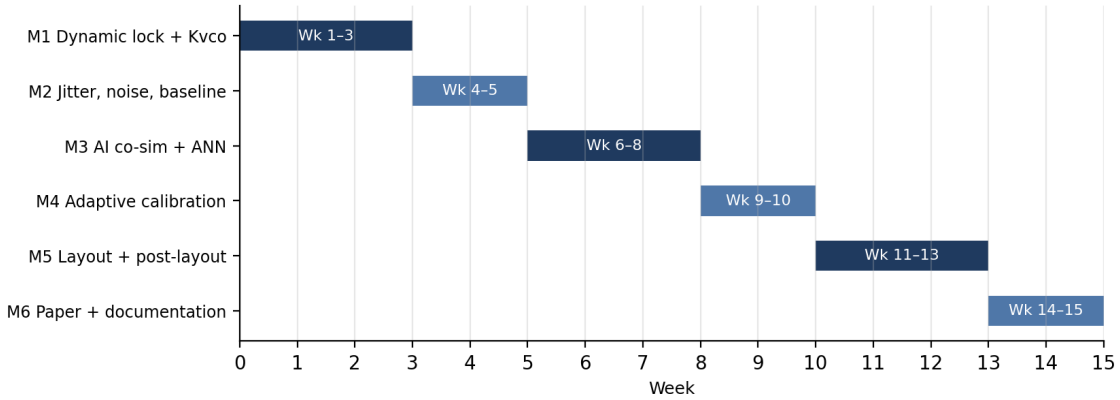


Figure 8. Fifteen-week plan.

Milestone	Deliverables	Weeks
M1. Dynamic lock and K_{VCO} characterization	Transient with toggling reference; K_{VCO} curve and linearity bounds; R1, C1, C2 locked	1–3
M2. Jitter, noise and baseline benchmark	pnoise standalone vs. closed loop; static phase offset and spur profile; baseline benchmark report	4–5
M3. AI co-simulation and model training	Python–Spectre IPC bridge; ANN pre-charge model trained; fast lock without cycle slips shown	6–8
M4. Adaptive calibration and optimization	Online mismatch cancellation; dynamic loop-filter zero tracking; reference spur reduction > 15 dB verified	9–10
M5. Layout, extraction, post-layout	DRC/LVS-clean layout with guard rings; RC extraction and re-simulation; PVT matrix	11–13
M6. Paper and documentation	Pre- vs. post-AI data synthesis; comparison graphs; 4-page conference paper	14–15

9. Risks and Mitigation

Several risks are worth naming now, because each has a cheap early check.

Risk	Why it matters	Mitigation
Baseline fails to lock cleanly	Everything downstream assumes a working loop	M1 comes first; retune filter from measured K_{VCO} before touching AI
Switch charge injection spoils the ANN hand-off	Error at hand-off can exceed pull-in range	Dummy switch, bottom-plate style switching, error budget set from pull-in range

Risk	Why it matters	Mitigation
Co-simulation too slow for RL training	Slow episodes limit training data	Behavioral PLL model for training, Spectre for validation; Verilog-A export route
4-bit trim limits spur floor	Resolution caps achievable suppression	Measure trim step effect early; add a bit or a dither if needed
AI overhead (area, power, supply noise)	Digital switching could raise spurs	Domain isolation, clock gating the AI core while locked, report its power in the results
Gains do not meet targets	Targets are ambitions, not guarantees	Report measured improvement honestly; the baseline-vs-AI comparison is publishable either way

10. Expected Outcomes and Conclusion

The table sets the planned comparison. Baseline numbers are estimates until Milestones 1 and 2 are complete; AI numbers are targets we will be held to.

Metric	Baseline 45 nm CPPLL (estimated)	AI-augmented target
Lock acquisition time	~1.5 to 5.0 µs (slew-limited pull-in)	< 250 ns (feedforward assist)
Integrated RMS jitter (from 10 kHz)	~1.2 to 2.5 ps (uncompensated)	< 350 fs (narrow-band optimized filter)
Reference spur	−30 to −40 dBc (uncalibrated mismatch)	< −58 dBc (background balancing)
Capture / pull-in range	Bandwidth dependent (~10% tuning span)	Extended over full LO range via ANN estimate
Design maturity	Transistor schematic-level bring-up complete	DRC/LVS-clean, post-layout-verified layout

In short, the baseline PLL is built and running. The coming work measures it properly, adds a feedforward ANN for fast lock, a learned loop tuner, and background mismatch calibration, and then proves the result through layout and corner analysis. If the targets are met, the outcome is a fast-locking, low-jitter local oscillator synthesizer and a solid conference paper; if they are only partly met, the measured before-and-after data still gives a clear, honest account of what AI can and cannot add to a 45 nm PLL.

Appendix A. Phase 1 Testbench Checklist

A short checklist to run before declaring the baseline characterized. Each item produces a number or a plot that goes into the final comparison.

#	Item	Output
1	Replace fixed vref with pulse source at f_{ref}; 20 ps edges; 45/50/55% duty	Locked transient for each duty case
2	Start from Vc(0) = 0 V, 0.6 V, 1.2 V; run 10–50 µs	T_p, T_L for each start
3	Step reference frequency up and down until acquisition fails	Hold-in and pull-in range
4	Plot V(up) − V(down) around edge collisions	Dead-zone / metastability check
5	Open the loop; sweep Vc 0–1.2 V in 10 mV steps	f_{out}(Vc) and K_{VCO}(Vc)
6	Compute R1, C1, C2 from average K_{VCO}; verify with AC analysis	Phase margin, loop bandwidth
7	Run pnoise 10 kHz–100 MHz and transient noise	Phase noise, RMS jitter
8	Measure locked spectrum at f_{ref} offset	Baseline reference spur

Appendix B. ANN Training Pipeline

The ANN is built in five steps, each one reproducible from saved data so results in the paper can be regenerated.

Step	Description
1. Data generation	Sweep N_{ch}, temperature, V_{DD} and corner; record steady-state Vc from the K_{VCO} curve and locked transients
2. Preprocessing	Normalize inputs; hold out corners for testing so generalization is measured on unseen conditions
3. Training	Small MLP (4-16-16-1, ReLU) in PyTorch; loss weighted so errors that exceed the pull-in budget cost more
4. Quantization	Quantization-aware training to 8-bit weights; check the accuracy against the DAC step size
5. Deployment	Export as lookup / fixed-point logic or Verilog-A model; validate inside the full PLL transient

Appendix C. Glossary

Term	Meaning
CPPLL	Charge-pump phase-locked loop
PFD	Phase-frequency detector
TSPC	True single-phase clock logic style
K_{VCO}	VCO tuning gain, rad/(s·V)
ΔI_{CP}	Charge-pump up/down current mismatch
ANN	Artificial (feedforward) neural network
RL	Reinforcement learning
PVT	Process, voltage, temperature
pnoise	Spectre RF periodic noise analysis
IPC	Inter-process communication
MOM	Metal-oxide-metal capacitor